

Durrmi CMS

Architecture Decision & Technical Design

Schema-driven, component-based content management platform

Document	Decision
Status	Proposed / Architecture Decision
Database	PostgreSQL
Media Storage	Azure Blob Storage
Primary Content API	GET /api/v1/pages/{slug}
Page Identity	Slug
Versioning	Required; immutable page versions
Draft Workflow	Not required
Section Scheduling	Required
Localization	English only for V1
API Separation	Single CMS API surface; authorization distinguishes admin operations

Prepared for Durrmi

1. Executive Summary

The proposed Durrmi CMS is a schema-driven, component-based content platform for managing configurable frontend pages. Pages are identified by a unique slug and are composed of ordered sections. Each section contains a root component and supports arbitrarily nested child components.

Reusable content is a first-class concept. Components, CTAs and media can be maintained in libraries and referenced from multiple pages. Component revisions are used to preserve historical page versions while allowing the current reusable component to evolve.

The CMS API resolves both static CMS content and registered dynamic data sources before returning the page response. This allows pages such as the blog page to use the same component model while sourcing article data from blog tables in the same PostgreSQL database.

2. Architecture Decisions

Decision Area	Decision	Rationale
Page identity	Unique slug	Stable frontend-facing key and simple routing model.
Page composition	Nested component tree	Supports flexible and future UI compositions without page-specific APIs.
Component model	Generic type + container type + content + children	Separates semantic component behavior from presentation.
Reusable content	Library instances referenced by ID	One reusable definition can be consumed by multiple pages.
Component styling	Frontend-controlled by containerType	Prevents arbitrary CMS CSS and keeps visual consistency in the frontend design system.
Media	Azure Blob Storage + metadata in PostgreSQL	Separates binary storage from CMS metadata.
Versioning	Immutable page versions plus component revisions	Enables rollback and historical consistency.
Workflow	No draft state; explicit publish/version operations	Matches current requirement while retaining history.
Scheduling	Section-level start/end scheduling	Allows campaigns and temporary sections without scheduling entire pages.
Dynamic content	Registered backend data sources resolved by CMS API	Keeps frontend contract simple and avoids arbitrary external API/SQL configuration.
Public content retrieval	GET /api/v1/pages/{slug}	One request returns the complete resolved page tree.
Database	PostgreSQL with JSONB	Strong relational integrity plus flexible component-specific content schemas.

3. Functional Scope

- Create, update and delete pages identified by slug.
- Create, delete, reorder and duplicate sections.
- Hide and unhide sections.
- Schedule sections for future activation and optional expiry.
- Build components using a generic, nested component-tree model.
- Reuse component instances, CTA instances and media across pages.
- Maintain component revisions so historical page versions remain stable.
- Maintain page version history and support rollback.
- Store rich text content.
- Support media such as images, icons and GIFs using Azure Blob Storage.
- Support configurable cards, flip cards and other UI component types.
- Resolve dynamic content from registered backend data sources.
- Support the blog page using the same page/component API.
- English-only content for V1.

4. High-Level Architecture

The system is divided into five logical layers:

- CMS Portal — administrative interface for page, section, component, CTA and media management.
- CMS API — single API surface responsible for management operations and public page composition.
- Content Store — PostgreSQL containing pages, versions, sections, component configuration, reusable content, CTAs, media metadata and blog data.
- Media Store — Azure Blob Storage for binary media.
- Data Resolver — backend layer that resolves registered dynamic data sources such as BLOG into the generic component response.

Logical flow:

```
CMS Portal → CMS API → PostgreSQL / Azure Blob / Data Resolver → Resolved Page JSON → Durrmi Frontend
```

5. Component Architecture

The CMS should not model individual frontend fields such as hero_header, hero_body or flip_card_front_header as database columns. Instead, the API uses generic content primitives.

5.1 Component contract

Recommended conceptual structure:

```
{
  "id": "cmp_001",
  "type": "CARD",
  "containerType": "FEATURE_CARD",
  "content": {
    "title": {
      "type": "RICH_TEXT",
      "value": "Fast Processing"
    },
    "description": {
      "type": "RICH_TEXT",
      "value": "Process requests quickly."
    },
    "icon": {
      "type": "MEDIA",
      "assetId": "asset_123"
    },
    "cta": {
      "type": "CTA",
      "id": "cta_123"
    }
  },
  "children": []
}
```

Commented [An1]: can keep only one: either type or containerType

5.2 Type vs containerType

Property	Responsibility	Examples
type	Semantic/UI component identity	CARD, FLIP_CARD, TEXT, IMAGE, CTA, CONTAINER
containerType	Frontend presentation/design variant	FEATURE_CARD, BLOG_CARD, HERO_CONTAINER, DARK_CARD

The CMS should not return arbitrary CSS such as padding, font size, border radius or colors. The frontend maps containerType to its design-system implementation.

5.3 Nested components

Components may contain child components recursively. A generic parent-child relationship avoids hard-coding a hierarchy such as Section → Cards → CTA and allows future compositions.

6. Domain Model

Entity	Purpose
Page	Website page identified by unique slug.
PageVersion	Immutable snapshot of page composition at a point in time.
Section	Ordered, independently visible/schedulable page region.
Component	Logical reusable component definition/identity.
ComponentRevision	Versioned component content/configuration.
ComponentInstance	Reference/use of reusable content in a page tree.
CTA	Reusable frontend action configuration such as label, URL and button type.
MediaAsset	Metadata for media stored in Azure Blob Storage.
DataSource	Registered dynamic data provider such as BLOG.
BlogArticle	Blog content stored in PostgreSQL.

7. PostgreSQL Data Model

The recommended approach is relational tables for core relationships with JSONB for component-specific content.

Table	Key Fields	Notes
pages	id, slug, name	slug is unique.
page_versions	id, page_id, version_number, status, created_by, published_at	Versions are immutable after publication.
page_sections	id, page_version_id, root_component_instance_id, display_order, is_visible, schedule_start_at, schedule_end_at	Scheduling is evaluated by CMS API.
components	id, type, container_type	Logical component identity.
component_revisions	id, component_id, revision_number, content_jsonb, data_source_jsonb	Preserves reusable content history.
component_instances	id, component_revision_id	Referenced from page trees.
component_relations	parent_instance_id, child_instance_id, display_order	Supports arbitrary nesting.
ctas	id, name, label, url, button_type, color, target	Reusable CTA library.
media_assets	id, blob_path, blob_url, file_name, content_type, size, alt_text	Binary data remains in Azure Blob.
data_sources	id, name, type, configuration_jsonb	Registry of allowed dynamic sources.
blog_articles	id, slug, title, body, image_id, published_at, category_id, status	Owned by the CMS DB.

The exact normalization can be refined during implementation. JSONB should be used for flexible component content, not for core referential relationships that require integrity, indexing or joins.

8. Versioning and Rollback

Versioning is required, but a Draft state is not. The recommended model is an immutable page-version snapshot with an explicit published version.

Example:

Page /products → Version 10 → Version 11 → Version 12 (published)

- Publishing creates or activates a new immutable page version.
- Previous versions remain readable for audit and rollback.
- Historical versions reference specific component revisions.
- Updating a reusable component creates a new component revision rather than mutating historical data.
- Rollback changes the current published page pointer to an existing page version rather than rewriting history.

9. Section Scheduling

Scheduling is intentionally section-level. A section may have no schedule, a start time, an end time, or both.

Section is active when:

```
is_visible = true
AND (schedule_start_at IS NULL OR schedule_start_at <= current_time)
AND (schedule_end_at IS NULL OR schedule_end_at > current_time)
```

The frontend should not implement this scheduling logic. The CMS API returns only currently active sections for the public page response.

10. Dynamic Data Resolution

The CMS supports components whose data is resolved from backend-owned sources. The Blog page is the first expected use case.

A component can conceptually specify:

```
{
  "type": "CARD_GRID",
  "containerType": "BLOG_CARD_GRID",
  "dataSource": {
    "type": "BLOG",
    "operation": "LIST",
    "parameters": {
      "category": "featured",
      "limit": 6
    }
  }
}
```

The CMS must use a registered data-source resolver rather than allowing arbitrary SQL or arbitrary external URLs to be configured by administrators.

```
BLOG → BlogDataResolver → BlogRepository → PostgreSQL
PRODUCT → ProductDataResolver → ProductRepository/API
```

Future sources can be added without changing the page API contract.

11. API Architecture

A separate Admin API is not required. A single versioned API surface can expose both management and public content operations, protected by authorization.

11.1 Primary frontend API

GET /api/v1/pages/{slug}

This is the canonical page-rendering API. It resolves page version, active sections, reusable content, media references and dynamic data before returning the response.

```
{
  "page": {
    "id": "page_001",
    "slug": "/blog",
    "version": 7
  },
  "sections": [
    {
      "id": "section_001",
      "order": 1,
      "component": {
        "id": "cmp_001",
        "type": "HERO",
        "containerType": "BLOG_HERO",
        "content": {
          "title": {
            "type": "RICH_TEXT",
            "value": "Durrmi Blog"
          }
        }
      },
      "children": []
    },
    {
      "id": "section_002",
      "order": 2,
      "component": {
        "id": "cmp_002",
        "type": "CARD_GRID",
        "containerType": "BLOG_CARD_GRID",
        "data": [
          {
            "id": "blog_001",
            "title": "Building scalable systems",
            "slug": "/blog/building-scalable-systems",
            "image": {
              "id": "asset_001",
              "url": "https://..."
            }
          }
        ]
      }
    }
  ]
}
```

11.2 Management endpoints

Method	Endpoint	Purpose
POST	/api/v1/pages	Create page.
PUT	/api/v1/pages/{pageId}	Update page metadata.
DELETE	/api/v1/pages/{pageId}	Delete page.
GET	/api/v1/pages/{pageId}/versions	List versions.
GET	/api/v1/pages/{pageId}/versions/{version}	Retrieve a specific version.
POST	/api/v1/pages/{pageId}/sections	Add section.
PUT	/api/v1/sections/{sectionId}	Update section.
DELETE	/api/v1/sections/{sectionId}	Delete section.
POST	/api/v1/sections/{sectionId}/duplicate	Duplicate section.
PUT	/api/v1/sections/reorder	Reorder sections.
POST	/api/v1/sections/{sectionId}/hide	Hide section.
POST	/api/v1/sections/{sectionId}/unhide	Unhide section.
PUT	/api/v1/sections/{sectionId}/schedule	Set section schedule.
POST	/api/v1/pages/{pageId}/publish	Publish a page version.
POST	/api/v1/pages/{pageId}/rollback	Rollback to a previous version.
GET/POST/PUT	/api/v1/components...	Component library management.
GET/POST/PUT	/api/v1/ctas...	CTA library management.
POST/GET	/api/v1/media...	Media library and upload metadata.

12. Blog Architecture

The blog is treated as a data domain inside the same PostgreSQL database, while its presentation is configured using normal CMS sections and components.

Example composition:

- Blog Hero → static CMS content.
- Featured Articles → CARD_GRID + BLOG data source.
- Latest Articles → CARD_GRID + BLOG data source.
- Reusable CTA → CTA library reference.

This avoids creating a special Blog Page API and ensures the frontend consumes the same page contract for both static and dynamic sections.

13. Media Architecture

- Binary files are uploaded to Azure Blob Storage.
- PostgreSQL stores media metadata and the blob reference.
- Components reference media assets by ID.
- The public API resolves the media metadata needed by the frontend.
- Media should support at minimum images, icons and GIFs.
- Alt text should be stored as CMS metadata for accessibility.

14. Frontend Integration

The frontend owns the actual rendering implementation. It maintains a component registry that maps the CMS component type to a frontend component.

```
const componentRegistry = {  
  HERO: Hero,  
  CARD: Card,  
  CARD_GRID: CardGrid,  
  FLIP_CARD: FlipCard,  
  IMAGE: Image,  
  TEXT: RichText,  
  CTA: CTA,  
  CONTAINER: Container  
};
```

The frontend can recursively render children. The CMS therefore remains presentation-agnostic while the frontend remains responsible for animation, responsive behavior and design-system styling.

15. Caching and Performance

- The public page API is read heavy and should be cacheable.
- Published page versions are immutable, which simplifies caching.
- A cache key can include a page slug and published version.
- Publishing a new version should invalidate the current page from cache.
- Dynamic data sources should have source-specific caching policies.
- Redis can be introduced if traffic or dynamic data volume warrants it; it is not mandatory for V1.

16. Security and Governance

- Management endpoints must require authenticated and authorized admin access.
- Public page retrieval must expose only published/active content.
- Dynamic data sources must be allow-listed and implemented server-side.
- Administrators must not be allowed to enter arbitrary SQL or arbitrary backend URLs as data sources.
- Azure Blob uploads should use controlled access and validated content types.
- Rich text must be sanitized/validated according to the frontend rendering strategy.
- Version and publish operations should be auditable with actor and timestamp metadata.

17. Key Trade-offs

Choice	Benefit	Trade-off
Generic JSONB content	Highly extensible component model.	Requires schema validation and careful indexing – Will not be indexed
Nested component tree	Supports arbitrary future UI compositions.	Tree traversal and editing logic are more complex – Will have to check with FE
Reusable linked instances	Changes propagate across consumers.	Requires revision/snapshot semantics for versioning – Can think if versioning is required
Single page API	Simple frontend integration and fewer round trips.	API must perform composition and dynamic-data resolution.
Frontend-controlled containerType	Consistent visual system.	Frontend and CMS component contracts must stay compatible.
Section-level scheduling	Flexible campaigns and temporary content.	Runtime API must evaluate time windows.

18. Architecture Decision Summary

Durrmi will implement a PostgreSQL-backed, schema-driven CMS in which a page is identified by slug and represented as a versioned tree of sections and reusable component instances. Components use generic content primitives and may be nested arbitrarily. Visual styling is controlled by containerType and the frontend design system.

Reusable components and CTAs are shared through library references. Component revisions preserve historical behavior. Section visibility and scheduling are resolved by the CMS API. Dynamic components use registered backend data sources, allowing blog content and future domains to be resolved into the same page response.

The canonical frontend contract is GET /api/v1/pages/{slug}, returning the fully resolved, currently active component tree. This keeps the frontend independent from the underlying CMS database and data source implementation.

19. Implementation Items to Finalize

- Exact component schema-definition mechanism and validation rules.
- Precise component instance/revision snapshot strategy.
- Rich-text storage format (e.g., structured JSON document model).
- Authorization roles and permissions for CMS operations.
- Media upload flow and Azure Blob access strategy.
- Blog article schema, category/tag model and publication states.
- Data-source resolver interface and supported operations/parameters.
- API error model, pagination and validation conventions.
- Cache/CDN strategy based on expected production traffic.

20. Recommended Refinements (Review Addendum)

The core architecture below is sound: the generic component tree, the JSONB-plus-relational split, immutable versioning, and allow-listed data sources all avoid common CMS pitfalls (hardcoded page schemas, CMS-controlled CSS, arbitrary SQL/URL data sources). The items below are refinements and open decisions surfaced during review, ranked by impact. They elaborate on, and should be read alongside, the open items already listed in Section 19.

20.1 Component schema validation (highest priority)

Section 19 already flags the schema-definition mechanism as open; this elevates it because it is the largest correctness risk in the design. Without a validation layer, “generic content primitives” becomes “any shape at write time,” and the frontend component registry will fail silently when a required field is missing or mistyped.

Recommendation: define a JSON Schema (or equivalent) per (type, containerType) pair, store it in a component_type_definitions table or config, and validate component_revisions.content_jsonb against it at write time in the CMS API — not just in the CMS Portal UI.

20.2 Preview path for the “no draft” workflow

“No draft workflow” is a reasonable V1 scope decision, but as written it implies editors cannot see a change before it goes live. page_versions.status already supports this: allow a DRAFT status alongside PUBLISHED, and add an authenticated preview endpoint that resolves a specific unpublished version.

```
GET /api/v1/pages/{slug}/preview?version={n} (admin-authenticated, bypasses the published-only filter)
```

20.3 Concurrency control on ordering and edits

display_order as a plain integer, edited by multiple admins concurrently, invites lost updates on reorder and edit operations. Recommend either optimistic locking (an updated_at/version check on section and component PUT requests, rejecting stale writes) or fractional/sparse ordering keys so reordering one item doesn't require rewriting every sibling's display_order.

20.4 Data-source resolver contract

Section 19 lists the resolver interface as an open item; the following is a proposed starting contract so it isn't designed from scratch mid-implementation:

```
resolve(operation: string, parameters: object, context: RequestContext)
→ { data: object[], cacheKey: string, ttlSeconds: number }
```

- Validate parameters per data source against a declared schema (mirrors 20.1).
- Enforce hard caps on caller-supplied parameters such as limit to prevent abusive queries.
- Return a resolver-supplied cache key/TTL so caching policy stays data-source-specific (consistent with Section 15).

20.5 Rich text storage format

Also listed as open in Section 19. Recommend committing early to a structured document model (e.g., a ProseMirror/Slate-style JSON tree) rather than raw HTML. This makes sanitization structural instead of regex-based, keeps rendering consistent across frontend surfaces, and makes future plain-text extraction (for search/indexing) straightforward.

20.6 Cache invalidation strategy

Section 15 notes that publishing a new version should invalidate the current page cache. Because published page versions are immutable, this can be made simpler and more robust than typical invalidation: cache resolved page responses permanently by the composite key (slug, version), and keep only the slug → current published version pointer on a short TTL (or uncached). Publishing then becomes “add a new immutable cache entry and flip a cheap pointer” rather than an invalidation event, with no risk of serving stale composed content.

20.7 Component revision reference semantics

Section 8 establishes that updating a reusable component creates a new revision rather than mutating history. What is not yet specified: when a section references a reusable component, does it pin a specific `component_revision_id` at save time, or always resolve to “latest revision” until the page is next published? This materially affects editor UX (silent live updates vs. explicit re-linking) and should be made an explicit decision in the domain model rather than left implicit in `component_instances.component_revision_id`.

20.8 Pagination on library endpoints

Section 19 flags API pagination generally as open. Recommend prioritizing this specifically for the library management endpoints (`/api/v1/components`, `/api/v1/ctas`, `/api/v1/media`) — these are the collections most likely to grow large in practice, and retrofitting cursor-based pagination after clients are built against an unpaginated list is more disruptive than including it from the first version.

20.9 Authorization role granularity

Section 16 requires authenticated and authorized admin access but does not define roles. Recommend at minimum separating the ability to edit content (EDITOR) from the ability to flip the published pointer (PUBLISHER). This is inexpensive to add now and considerably more disruptive to retrofit once editorial workflows are established.

Note: JSONB indexing was raised during review and confirmed not required for V1; no indexing strategy is prescribed here.